

Deep Neural Network Surrogates for Approximate Bayesian Computation: A Simulation Study for Mutation-Rate Estimation in Markov Branching Processes

Sachin Letchumanan^{1*}, Hongxiao Zhu¹ and Xiaowei Wu¹

^{1*}Department of Statistics, Virginia Tech, Blacksburg, VA, 24061, USA.

*Corresponding author(s). E-mail(s): sachin.letchumanan@gmail.com;
Contributing authors: hongxiao@vt.edu; xwwu@vt.edu;

Abstract

Approximate Bayesian computation (ABC) makes likelihood-free inference possible for costly stochastic simulators, but its per-iteration cost puts a full posterior analysis out of reach once the simulator is slow, as it is for the branching-process simulators used to estimate mutation rates from Luria-Delbrück-style fluctuation experiments. A standard remedy substitutes a Gaussian-process surrogate for the simulator inside the acceptance step of an ABC Markov chain Monte Carlo (MCMC) sampler, but a cubic fitting cost caps how much simulator output the surrogate can use, and that ceiling bites hardest exactly where relieving it is needed most. We ask whether a neural-network surrogate, whose query cost does not grow with training-set size, can perform the same substitution, and when doing so helps rather than hurts. We build a heteroscedastic feedforward surrogate, calibrated by split-conformal prediction, and evaluate it in two simulation studies.

In a one-dimensional benchmark, the network matches or exceeds the Gaussian process’s accuracy and returns tighter, still-calibrated intervals. The reason is architectural, not incidental: the network’s heteroscedastic head learns an input-dependent predictive variance directly, rather than the single fixed noise term a Gaussian process supplies, a distinction that matters because the true replicate-to-replicate noise in this problem is far from constant across the parameter range, varying up to 184-fold and correlating at -0.82 with the response itself in the two-stage model, a shape no homoscedastic noise term can represent. Because its query cost stays flat as the training design grows, rather than scaling with it as the Gaussian process’s does, the network’s speed advantage widens as that design grows, and it is consistently 68 to over 2,000 times faster than exact ABC-MCMC. In a second, three-parameter study, an amortized neural posterior estimator trained on the same simulation budget beats every surrogate tested, and the exact simulator-based sampler itself, on the model’s one well-identified parameter. These wins are conditional rather than universal: once both surrogates already sit at the data’s accuracy ceiling, the network’s edge over the Gaussian process disappears entirely, and the comparison is instead governed by which parameters the model can identify at all, a limitation we measure directly rather than assume away, and discuss in full in the sections that follow.

Together, the studies show that a neural surrogate is worth deploying only where surrogate error still binds accuracy, not by default; that an amortized neural posterior estimator is a useful complement for pinning down well-identified parameters at the same simulation budget rather than a general substitute

for sampling; and that a learned surrogate’s calibration must be verified by measuring posterior coverage directly, since standard diagnostics do not guarantee it. Code, data, and a reproducible pipeline accompany the paper so both claims can be checked and reused.

Keywords: approximate Bayesian computation; likelihood-free inference; surrogate modeling; conformal prediction; heteroscedastic regression; simulation study; Markov branching process

1 Introduction

Approximate Bayesian computation (ABC) makes Bayesian inference possible for stochastic models whose likelihood is intractable but which can be simulated forward (Beaumont et al, 2002; Marjoram et al, 2003; Marin et al, 2012). The basic ABC-MCMC sampler (Marjoram et al, 2003) accepts a proposed parameter θ' in proportion to how closely a simulated summary statistic $S(\theta')$ matches the observed statistic S_{obs} . That simplicity carries a price: every iteration costs one or more forward simulations. When the simulator is expensive, this per-iteration cost dominates the runtime of the entire analysis and can put a full posterior exploration out of reach altogether.

Lu et al (2023) study this problem for a two-type Markov branching process (MBP) model of microbial mutation, used to estimate the per-division mutation probability from Luria–Delbrück-style fluctuation experiments. Their exact simulator (their Algorithm 2) simulates the branching process cell by cell, and a single call takes anywhere from a fraction of a second to several minutes depending on the parameter regime. To make ABC-MCMC tractable, they apply *GPS-ABC* (Meeds and Welling, 2014): fit a Gaussian process (GP) regression surrogate to a modest design of simulator outputs, then substitute the GP’s predictive mean and variance for the simulator inside the MCMC acceptance step. Every iteration becomes a GP prediction rather than a simulator call, which is where their reported two-to-three-order-of-magnitude speedup comes from.

The Gaussian process, however, brings a scaling limitation that is well known and, for this application, binding: fitting an n -point GP costs $O(n^3)$ and querying it costs $O(n)$ per point, for the naïve dense implementation used in practice at this problem size. Two consequences follow, and the second is the more damaging. The first is a ceiling on the design; Lu et al (2023) report being limited to

roughly 50 points in the one-dimensional case and 1,500 points (growing further via a sequential rule) in their four-parameter extension, for exactly this reason. The second is that the ceiling bites hardest where relieving it is most expensive. They note that GPS-ABC can in fact be *less* accurate than plain ABC-MCMC when the surrogate’s design is built from the expensive exact simulator, precisely because so few exact-simulator calls can be afforded for that design. A cubic fitting cost is therefore not merely an inconvenience of implementation; it propagates into the accuracy of the inference it was introduced to accelerate. This motivates the question the present paper asks:

Can a neural network surrogate replace the Gaussian process inside this same ABC-MCMC procedure, and if so, under what conditions does it help?

A feedforward network carries neither cost. It has no cubic fitting term, and its query cost is *flat* in the size of its training set, so it can be trained on substantially more simulator output than a GP can afford at a matched computational budget, an advantage that does not depend on input dimension and should, in principle, persist as the parameter and summary-statistic dimensions grow past the three tested in Study II below, though this paper tests that only at the two scales it reports rather than along a dimension sweep. Neither property is free, however. A GP’s predictive variance is a direct consequence of the Gaussian-process prior and requires no separate calibration step, whereas a neural network’s predictive uncertainty must be constructed and calibrated explicitly if it is to play the same role inside the ABC acceptance kernel. That requirement, rather than the regression itself, is what makes the substitution non-trivial, and it is what Section 3 is principally about.

Contributions. This paper makes six contributions. We build a heteroscedastic neural-network surrogate, calibrated by split-conformal prediction (Lei et al, 2018), that substitutes for the Gaussian process inside Lu et al’s ABC-MCMC sampler with

no other change to the inference algorithm. We run a one-dimensional simulation study reproducing Lu et al’s constant-mutation-rate benchmark directly, comparing four estimators (method of moments, maximum likelihood, exact ABC-MCMC, and GPS-ABC) against the neural surrogate on estimation accuracy, credible-interval length, coverage, and wall-clock cost. We then run a second simulation study on the two-stage mutation model that is the original paper’s actual multi-parameter contribution, estimating (p_1, p_2, τ) jointly, and we report, honestly and without tuning it away, that the neural surrogate’s advantage does *not* carry over to that regime; we pair that report with a measurement of the data’s irreducible noise floor that explains why. In that same second study we add an amortized neural posterior estimator (NPE) trained on the same simulation budget (scoped here rather than to the first study because this is the one setting with both a well-identified and weakly-identified parameter jointly estimated, the contrast that makes the amortized-vs-surrogate question informative; Section 5.6 gives the full reasoning) and find an asymmetry: NPE beats every surrogate tested, and the exact simulator-based sampler itself, on the well-identified parameter, resolved beyond replicate noise in most comparisons, but does no better than any other method on the two the model itself cannot pin down. Finally, rather than assume it, we test whether a convolutional or recurrent architecture improves on the architecture-agnostic feedforward surrogate in that same regime, motivated by where such architectures are known to help elsewhere in the simulation-based-inference literature, and find that neither does.

Related work. GPS-ABC (Meeds and Welling, 2014) is one instance of a broader line of work using Gaussian processes to reduce simulator calls in likelihood-free inference: Wilkinson (2014) models the ABC log-likelihood surface directly rather than the acceptance kernel, Gutmann and Corander (2016)’s BOLFI frames the design problem as Bayesian optimization over where to place the next expensive simulation, and Järvenpää et al (2021) extends this to noisy, parallel-batch likelihood evaluations. In this same journal, Järvenpää and Corander (2023) extends the surrogate-based line of work from parameter inference to prediction under an intractable model. All four inherit the same $O(n^3)$ fitting cost this paper’s Introduction opens with; none substitutes a neural surrogate for

the GP, which is the specific substitution this paper studies. A separate and now much larger literature bypasses the GP-versus-network choice entirely by learning the posterior (or likelihood, or likelihood ratio) directly with a neural density estimator trained on simulator draws – *neural* simulation-based inference (Papamakarios and Murray, 2016; Greenberg et al, 2019; Cranmer et al, 2020), implemented in practice through toolkits such as `sbi` (Tejero-Cantero et al, 2020). These methods are *amortized*: training cost is paid once, and posteriors for new data are then produced without any additional simulation or MCMC, which GPS-ABC, DNN-ABC and exact ABC-MCMC alike cannot offer, since each requires a fresh MCMC run per observed data set. Section 5.6 adds an amortized neural posterior estimator, trained on the same simulation budget, to the comparison this paper otherwise restricts to the surrogate-inside-MCMC design, to make that trade-off concrete rather than assumed, scoped to Study II rather than Study I for the reason given in the Contributions paragraph above. The comparison is also motivated by a finding from that literature that bears directly on this paper’s own results: Talts et al (2018)’s simulation-based calibration protocol and Hermans et al (2022)’s large-scale audit of neural SBI methods both report that posterior approximations, neural or otherwise, routinely pass weaker regression-level diagnostics while failing to cover the true parameter at the stated rate once checked directly. A concurrent finding in this same journal is specific to NPE rather than SBI broadly: Fan and White (2026), fitting exponential random graph models, report that an overly narrow NPE proposal can yield a biased, truncated posterior that nonetheless reproduces the observed summary statistics closely, coverage failing silently, in a different model family, for the same class of estimator Section 5.6 tests here. Section 4.4’s distinction between the surrogate’s own regression coverage and the ABC posterior’s coverage is an instance of exactly this failure mode, measured here for a GP/DNN-surrogate-based sampler rather than a neural density estimator.

The paper proceeds in that order. Section 2 reviews the MBP model and GPS-ABC. Section 3 develops the general neural surrogate framework.

Sections 4 and 5 report the constant-rate and two-stage simulation studies. Section 6 discusses implications and limitations, and Section 7 describes the reproducible pipeline accompanying this paper.

2 Background: ABC for the Markov Branching Process Model

2.1 The mutation model and the observed statistic

Following Lu et al (2023), a population initiated by a single wild-type cell grows for a fixed observation period under a two-type Markov branching process (MBP). Three quantities govern it. The first is the per-division mutation probability p , the rate at which a dividing wild-type cell produces a mutant daughter instead of a wild-type one; it is the estimand throughout this paper. The second is the division-rate parameter a , which sets the overall time scale of growth. The third is the mutant relative growth rate δ , the ratio of the mutant lineage’s growth rate to the wild type’s. We refer the reader to Lu et al (2023) for the full stochastic-process construction and for their Algorithm 2 (an exact, cell-by-cell simulator, slow) and Algorithm 4 (a fast approximate simulator using distributional shortcuts), both of which we reimplement in Python and validate directly against their original R/MATLAB output (Section 3.3).

An experiment consists of J independently plated parallel cultures, each grown from a single wild-type cell for the same fixed time. Culture j yields a final wild-type count Z_j and mutant count X_j . Following Lu et al, we work with the single scalar summary statistic

$$\bar{d} = \frac{1}{J} \sum_{j=1}^J \sqrt{X_j/Z_j},$$

which is well behaved across the many orders of magnitude spanned by p and is the statistic used throughout their ABC procedure. One further transformation is worth stating explicitly, because every regression in this paper inherits it: we model $\log_{10} \bar{d}$ rather than $\bar{d}$ itself. The reason is that $\log_{10} \bar{d}$ varies smoothly and close to linearly in $\theta \equiv \log_{10} p$ (their Figure 2), and a target that is smooth and nearly linear in its input is one

that both Gaussian-process and neural-network regression fit considerably better.

2.2 ABC-MCMC and GPS-ABC

Write $\theta = \log_{10} p$ for the (log-scale) parameter of interest. Given an observed statistic $\bar{d}_{\text{obs}}$, classical ABC-MCMC (Marjoram et al, 2003) targets the approximate posterior

$$\pi_{\epsilon}(\theta \mid \bar{d}_{\text{obs}}) \propto \pi(\theta) \mathbb{E}_{S \sim F_{\theta}} [K_{\epsilon}(\bar{d}_{\text{obs}}, S)],$$

by drawing a single summary S from the simulator at each proposed θ and weighting it with a smoothing kernel K_{ϵ} of bandwidth ϵ that down-weights simulated statistics far from $\bar{d}_{\text{obs}}$. The "exact" baseline used throughout this paper is not this literal kernel average: it is Wood’s (Wood, 2010) *synthetic likelihood*. At each proposed θ' , we draw n_s pilot simulations from $F_{\theta'}$, take their sample mean $m(\theta')$ and variance $v(\theta')$, and score $\bar{d}_{\text{obs}}$ against a matched Gaussian,

$$\hat{\pi}_{\epsilon}(\bar{d}_{\text{obs}} \mid \theta') = \mathcal{N}(\bar{d}_{\text{obs}}; m(\theta'), \epsilon^2 + v(\theta')).$$

This is a deliberate choice, not an approximation of convenience: it is exactly the likelihood form Section 3’s surrogates use, with a predicted variance $\sigma_{\phi}(\theta')^2$ in place of the pilot-simulation variance $v(\theta')$, so that every column of Tables 2–5 shares one likelihood and differs only in how $(m(\theta), v(\theta))$ is obtained. Algorithm 1 states the shared Metropolis–Hastings sampler (following Lu et al): a truncated-exponential prior on θ and a truncated-normal random-walk proposal, wrapped around this likelihood.

Algorithm 1 ABC-MCMC for $\theta = \log_{10} p$ (Metropolis–Hastings, synthetic likelihood)

- 1: Initialize θ_0 ; set tolerance ϵ , simulator-repeat count n_s .
- 2: **for** $t = 1, \dots, T$ **do**
- 3: Propose $\theta' \sim q(\cdot \mid \theta_{t-1})$ (truncated-normal random walk)
- 4: Obtain $(m(\theta'), v(\theta'))$: either (a) simulate n_s times from $F_{\theta'}$ and take their sample mean and variance (synthetic likelihood; Wood, 2010), or (b) evaluate a surrogate’s predicted mean and variance $(\mu_\phi(\theta'), \sigma_\phi(\theta')^2)$ (Section 3)
- 5: Compute acceptance probability

$$\alpha = \min\left(1, \frac{\pi(\theta') q(\theta_{t-1} \mid \theta')}{\pi(\theta_{t-1}) q(\theta' \mid \theta_{t-1})} \cdot \frac{\hat{\pi}_\epsilon(\bar{d}_{\text{obs}} \mid \theta')}{\hat{\pi}_\epsilon(\bar{d}_{\text{obs}} \mid \theta_{t-1})}\right)$$

- 6: Set $\theta_t = \theta'$ with probability α , else $\theta_t = \theta_{t-1}$
 - 7: **end for**
 - 8: **return** posterior sample $\{\theta_t\}_{t=B+1}^T$ after discarding B burn-in draws
-

Under this shared likelihood, step 4(a) draws n_s fresh simulations from the true process at θ' to estimate (m, v) , the expensive step. *GPS-ABC* (Meeds and Welling, 2014), applied by Lu et al (2023) to the branching-process model studied here, instead fits a Gaussian process to a design $\{(\theta_i, \bar{d}_i)\}_{i=1}^n$ of simulator outputs *once*, before sampling begins, and at every iteration uses step 4(b): the GP’s posterior predictive mean $\mu_{\text{GP}}(\theta')$ and variance $\sigma_{\text{GP}}(\theta')^2$ take the place of $(m(\theta'), v(\theta'))$ directly, no pilot simulation required. This is the mechanism by which the GP’s predictive variance enters the acceptance probability directly, and it sets the standard that any replacement surrogate must meet. What is required is not merely an accurate mean but a well-calibrated variance, because the two failure modes are asymmetric and both corrupt the posterior: an overconfident σ inflates the acceptance rate at parameter values the surrogate does not actually know well, while an underconfident one over-rejects. A surrogate that predicts the response perfectly and its own uncertainty badly is not a usable substitute here.

The GP’s fitting cost is $O(n^3)$ in the design size n and its per-query cost during MCMC sampling is $O(n)$, both because the predictive mean

and variance require solving a linear system involving the full training kernel matrix. Lu et al (2023) report design sizes of roughly 50 points in the one-dimensional constant-rate case and 1,500 initial points (via a Latin-hypercube design, growing further whenever a decision-error threshold is exceeded) in a four-parameter piecewise-rate extension, and note that GPS-ABC trained on the exact simulator can be *less* accurate than uncorrected ABC-MCMC in some one-dimensional configurations, plausibly a consequence of the small design the cubic cost permits when the design points themselves come from the expensive exact simulator.

3 A Neural-Network Surrogate for ABC

We replace the Gaussian process in step 4(b) of Algorithm 1 with a neural network f_ϕ trained to predict both the mean and the variance of $\log_{10} \bar{d}$ as a function of the parameter(s) of interest. The substitution is deliberately minimal: the sampler, the acceptance ratio, and the convolution with ϵ^2 are all exactly those of GPS-ABC, so that any difference in the results that follow is attributable to the surrogate and to nothing else. We refer to the resulting procedure as *DNN-ABC*. The two subsections below supply the two things the network needs in order to stand in for the GP: an input-dependent predictive variance, and a guarantee that the variance is calibrated.

3.1 Heteroscedastic regression

f_ϕ has two output heads, $\mu_\phi(x)$ and $\log \sigma_\phi^2(x)$, where x is $\theta = \log_{10} p$ alone (Section 4) or the parameter vector $(\log_{10} p_1, \log_{10} p_2, \tau)$ (Section 5); in both studies the surrogate is a direct function of the parameters being inferred, with no derived features. Both heads are trained jointly by minimizing the Gaussian negative log-likelihood

$$\mathcal{L}(\phi) = \frac{1}{n} \sum_{i=1}^n \left[\frac{(y_i - \mu_\phi(x_i))^2}{2 \sigma_\phi^2(x_i)} + \frac{1}{2} \log \sigma_\phi^2(x_i) \right], \quad (1)$$

where $y_i = \log_{10} \bar{d}_i$. The difference from a single-output network trained under squared-error loss is the point of the construction. Equation 1 targets an *input-dependent* predictive variance directly, the

network’s analogue of the GP’s posterior variance, and Section 4 shows that this is not a refinement but a requirement: the true replicate-to-replicate noise in $\log_{10} \bar{d}$ is far from homoscedastic across the parameter range, and a Gaussian process carrying a single noise term cannot represent that shape at all.

3.2 Split-conformal calibration

Training a variance head is not the same as calibrating one. The head produced by Equation 1 is an approximation to the true conditional variance learned from a finite training set, and nothing in that objective guarantees it is well calibrated in finite samples. Since Section 2.2 established that the acceptance step consumes this quantity directly, an uncalibrated head is a defect the sampler will inherit. We therefore apply split-conformal calibration (Lei et al, 2018) to correct it. After training on a training split, we form calibration scores on a held-out calibration split of size m ,

$$s_i = \frac{|y_i - \mu_\phi(x_i)|}{\sigma_\phi(x_i)}, \quad i = 1, \dots, m,$$

and set the calibration factor $\hat{c}$ to the $\lceil (m+1)(1-\alpha) \rceil / m$ empirical quantile of $\{s_i\}$, the finite-sample correction of Lei et al (2018). The calibrated $(1-\alpha)$ predictive interval for a new input x is then $\mu_\phi(x) \pm \hat{c} \sigma_\phi(x)$. Provided the calibration and test points are exchangeable, this construction guarantees marginal coverage of at least $1-\alpha$ in finite samples, without any distributional assumption on f_ϕ ’s errors (Lei et al, 2018). We use $\alpha = 0.05$ throughout, and refer to this coverage guarantee as the surrogate’s *regression calibration*, to distinguish it explicitly from the empirical coverage of the downstream ABC posterior’s credible intervals, a materially different quantity with no equivalent theoretical guarantee (Sections 4.2 and 5.8 report both, and the distinction between them is central to the honest limitation reported in Section 5.8).

The calibrated $\hat{c} \sigma_\phi(x)$ is used as the surrogate’s predictive standard deviation both when reporting regression calibration and when plugging $\sigma_\phi(\theta')$ into the ABC-MCMC acceptance kernel of Algorithm 1, so that the same calibrated uncertainty that is validated in Sections 4.2 and 5 is what actually enters the sampler.

3.3 Ground-truth data and simulator validation

In both studies, ground truth is generated by the exact/slow MBP simulator (Lu et al’s Algorithm 2), re-implemented in Python from the original R/MATLAB code. A ported simulator is an assumption until it is checked, and every number in this paper rests on it, so the port is validated in two ways before any surrogate is trained: the closed-form plating-time equation is solved and compared against the R implementation’s solver output (agreement to 3×10^{-5}), and the simulator’s simulated $\bar{d}$ means are compared against the committed ground-truth CSV at matched grid points (Sections 4 and 5 report the resulting agreement for each study).

3.4 Simulation design and reporting

We report both simulation studies following the structure recommended by Morris et al (2019): *aims*: compare five estimators (MOM, MLE, exact ABC-MCMC, GPS-ABC, DNN-ABC) in Study I, and the two surrogate methods (GPS-ABC, DNN-ABC) in Study II, where the constant-rate MOM and MLE estimators do not apply, on point accuracy, interval precision and coverage, and computational cost, under known ground truth; *data-generating mechanisms*: the validated MBP simulator of Section 3.3, at the parameter grids specified in Sections 4.3 and 5.6; *estimands*: the mutation probability p (via $\theta = \log_{10} p$) in Study I, and the two-stage parameters (p_1, p_2, τ) jointly in Study II; *methods*: the estimators of Section 2.2 and 3, all evaluated on identical simulated data sets within a cell; and *performance measures*: MSE and normalized RMSE of $\hat{p}$, mean 95% credible-interval length, empirical coverage, and wall-clock time per 100 MCMC iterations.

Each cell of each study is evaluated over R independent replicated data sets ($R = 40$ in Study I, $R = 16$ per truth triple in Study II), and, following Morris et al’s recommendation, we accompany every MSE comparison with its Monte Carlo standard error (MCSE), computed as the usual standard error of a sample mean applied to the per-replicate squared errors: writing $e_r = (\hat{p}_r - p)^2$ for replicate $r = 1, \dots, R$ under a given method and cell, $\widehat{\text{MSE}} = R^{-1} \sum_r e_r$ has $\widehat{\text{MCSE}} = \text{sd}(e_r)/\sqrt{R}$. For the head-to-head

comparison between the two surrogates in each cell, we report the gap $\widehat{\text{MSE}}_{\text{GPS}} - \widehat{\text{MSE}}_{\text{DNN}}$ in units of its own combined Monte Carlo standard error $\sqrt{\text{MCSE}_{\text{GPS}}^2 + \text{MCSE}_{\text{DNN}}^2}$ (a Δ/SE column in Table 2, and Section 5.6 for Study II); this is a descriptive diagnostic of whether a point comparison is resolved at this replicate count, not a formal hypothesis test, and we use it throughout to avoid over-interpreting small numerical differences as the replicate count varies across cells and between the two studies ($R = 40$ versus $R = 16$).

4 Simulation Study I: One-Dimensional Constant Mutation Rate

The first study reproduces Lu et al’s original benchmark exactly: $a = \delta = Z_0 = 1$ held fixed, and only p varying. Ground truth is 101 log-spaced values of p with $\log_{10} p \in [-8, -2]$, ten replicates each (1,010 rows total), $J = 100$. We split *by replicate* rather than by grid point, which is what keeps the comparison honest: every value of p then appears in every split, so a split tests generalisation to fresh noise rather than to an unseen region of the parameter axis, and no grid point can leak from training into evaluation. Replicates 1–5 are used for training (505 rows), 6–8 for conformal calibration and early stopping (303 rows), and 9–10 are held out for test-only evaluation (202 rows). The GPS-ABC baseline is fit on the identical replicates 1–8, so that both surrogates see the same data budget; neither ever sees the test replicates.

The ABC-MCMC comparison below (Section 4.3) evaluates observed data sets simulated at $J \in \{10, 50, 100\}$ cultures, not only the $J = 100$ the ground-truth grid is generated at. Both surrogates are trained separately for each J rather than once at $J = 100$ and queried at every J : for $J \in \{10, 50\}$, a J -culture $\bar{d}$ is reconstructed from the same underlying simulated data by averaging a random J -subset of the 100 recorded per-culture values (five independent subsets per row, to keep the resampled training set a reasonable size), so this requires no new simulator calls. Section 4.4 explains why this matters and reports what training a single $J = 100$ surrogate and querying it away from that condition actually does to posterior coverage.

4.1 Architecture, and the one choice that mattered

The network is a shallow funnel multilayer perceptron: input $\theta = \log_{10} p$, two GELU-activated dense layers, and the two linear output heads of Section 3.1. Its final width, 32 then 16 (626 parameters), is the outcome of a two-stage search: an activation/normalisation sweep at a 128–64 baseline, then a capacity sweep of that baseline once the activation question was settled. Full tables for both stages are in Supplementary Material S1.

The activation/normalisation sweep’s one decisive finding is BatchNorm (Ioffe and Szegedy, 2015): an otherwise-reasonable 128–64 network carrying it and dropout fits *eleven times worse* than the same network without them. The mechanism is specific to this problem rather than general. BatchNorm normalises per-minibatch statistics, useful in deep classification networks; but on a smooth scalar regression over a one-sided domain (the θ -grid has a hard edge at -8) it injects batch-dependent noise and biases predictions near that boundary. What closes the gap to the GP is therefore not any choice of activation function, but the removal of a component most practitioners would have included by default. A follow-up ablation isolating only the activation function (same architecture, no BatchNorm, three seeds) found ReLU, SiLU (Elfwing et al, 2018), GELU (Hendrycks and Gimpel, 2016), and tanh within 7% of one another: a statistical tie, with ReLU marginally ahead. Because accuracy does not separate them, GELU is chosen on structural grounds instead: smoothness, which matters should the sampler later be adapted to a gradient-based scheme, and immunity to the dead-neuron failure mode, at no measurable accuracy cost. A second search round using 7-network deep ensembles did not improve on the single chosen network, so the simpler model is retained throughout.

The network is trained with an Adam optimizer under early stopping on calibration-split negative log-likelihood, weight decay 10^{-5} , no dropout (early stopping and weight decay already control overfitting on this smooth curve; dropout’s injected noise measurably hurt the fit in preliminary runs). **Why convolutional and recurrent architectures are not tested here.** Study II (Section 5.7) reports a controlled comparison of feedforward, convolutional, and recurrent surrogates on the three-input two-stage problem. We do not repeat

that comparison here because it would be vacuous: with a single scalar input $\theta = \log_{10} p$, a 1-D convolution has no neighbouring position to convolve over and a recurrent cell has no second time step to recur into, so both architectures collapse mathematically to the already-tested single-layer linear map.

Is the two-layer 128–64 network actually needed? No. A width/depth sweep from the 128–64 baseline down to a linear control (three seeds each, screened at this project’s own quick-check settings then confirmed at the paper’s full scale: 40 replicates, the full $p \times J$ grid, 600 MCMC iterations, with Monte Carlo standard errors attached exactly as in Section 3.4) finds 32–16 the best point estimate on both curve fit and the downstream task, at $14\times$ fewer parameters than 128–64, though this specific margin is not itself distinguishable from Monte Carlo noise at this replicate count, so what is established is that the smaller network is not worse, not that it is provably better. Every candidate from 8 to 128–64 parameters beats the GP baseline by a wide, decisive margin in every cell; only a purely linear control shows a clear cost, confirming some nonlinearity is required, just not much capacity beyond that. Full per-candidate tables are in Supplementary Material S1.

We redeploy on the strength of this comparison. 32–16 is the best point estimate everywhere it can be compared to 128–64 and strictly better on curve fit, at $14\times$ fewer parameters; no analysis in this paper can show a cost to making the change, and Occam’s razor settles a comparison this evenly matched in favour of the smaller model. The deployed network is 32–16; Tables 2–4 and every figure in this study are reported for it, retrained from the same data splits and re-evaluated at the same paper scale (including the exact-simulator baseline). The 128–64 configuration is retained in Supplementary Material S1 as the architecture the activation and BatchNorm search was actually run against, not as the deployed model.

4.2 Regression calibration

Table 1 reports the surrogate’s fit and split-conformal coverage on each data split.

The held-out test-set coverage of 0.9554 (193 of 202 points) is within sampling noise of the nominal 0.95, consistent with the finite-sample guarantee

of Lei et al (2018). Figure 1 shows the calibrated 95% band against the test data and the resulting test-set parity, and compares the surrogate’s input-dependent predictive standard deviation to the GP’s flat, homoscedastic one.

4.3 ABC simulation results

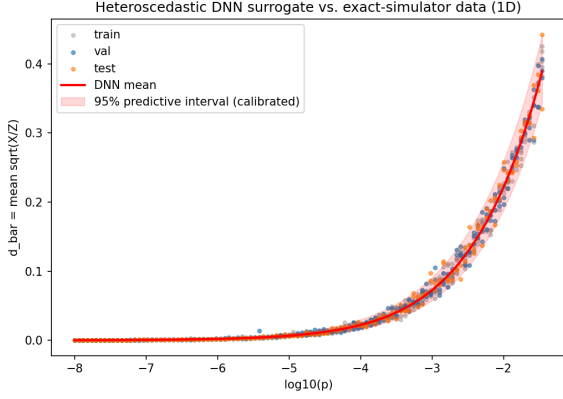
We ran the full ABC-MCMC / GPS-ABC / DNN-ABC comparison at 40 independent replicated data sets per cell, 600 MCMC iterations (250 burn-in), $n_s = 6$ simulator repeats per exact-ABC iteration, prior $\theta \sim \text{TruncExp}$ on $[-5, -2]$ (bounded so that the exact simulator inside the ABC-MCMC loop remains computationally feasible; both surrogates are trained on the wider $[-8, -2]$ range and only queried in-range), on the grid $p \in \{10^{-4}, 10^{-3}, 10^{-2}\} \times J \in \{10, 50, 100\}$. Method of moments (MOM) and maximum likelihood (MLE) estimators follow Lu et al’s closed-form expressions and serve as classical, non-ABC baselines.

Two readings of this table must be kept apart, and conflating them is the error the Δ/SE column exists to prevent. DNN-ABC’s point estimate is numerically lower than GPS-ABC’s in five of the nine cells and higher in the other four. But nominally lower is not the same as reliably better, and the Δ/SE column shows that none of these differences, in either direction, is distinguishable from noise at $R = 40$ replicates: the largest magnitude reached anywhere in the table is 1.44 ($p = 10^{-2}$, $J = 50$, nominally favouring GPS-ABC), short of the ~ 2 threshold used throughout this paper. This is a near-exact statistical tie in every tested cell, consistent with a GP fit on a comparably sized design already being close to optimal across this parameter range once both surrogates are trained on the J -specific data described in Section 4; Section 4.4 returns to why training each surrogate to know its own culture count matters for more than point accuracy. Both surrogates match, and in several cells slightly improve on, the exact ABC-MCMC baseline, and all ABC-based methods substantially outperform the classical MOM/MLE estimators, reproducing Lu et al’s central finding that ABC dominates the classical estimators for this model.

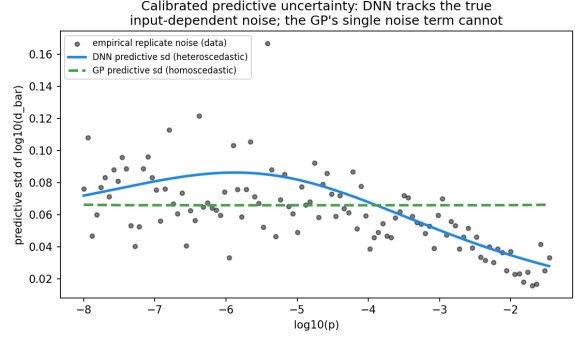
DNN-ABC’s interval is shorter than GPS-ABC’s in all nine cells, 21% tighter on average and up to 46% tighter, without any loss of point accuracy (Table 2), though the margin varies widely,

Table 1: One-dimensional surrogate: fit and 95% regression coverage by split.

Split	n	MSE($\log_{10} \bar{d}$)	MAE($\log_{10} \bar{d}$)	95% coverage
Train	505	0.00424	0.0515	0.964
Calibration	303	0.00533	0.0538	0.954
Test	202	0.00434	0.0525	0.9554



(a) Calibrated 95% predictive band



(b) Input-dependent uncertainty vs. the GP

Fig. 1: One-dimensional surrogate calibration. Left: the calibrated 95% predictive band tracks the response curve across the full range of θ . Right: the network’s heteroscedastic predictive standard deviation (which enters the ABC acceptance kernel directly) tracks the true input-dependent replicate noise, while the GP’s single homoscedastic noise term is flat by construction.

from negligible (0.1% at $p = 10^{-4}$, $J = 100$) to substantial (46% at $p = 10^{-2}$, $J = 100$), and against exact ABC-MCMC specifically DNN-ABC is narrower in seven of nine cells rather than all nine (exact ABC-MCMC is marginally shorter at $J = 10$ for both $p = 10^{-4}$ and $p = 10^{-2}$). Whether DNN-ABC’s advantage over GPS-ABC is genuine gained precision or overconfidence is not settled by interval length alone. Section 4.4 measures the posterior coverage this comparison needs, and the answer depends on J .

DNN-ABC runs at a flat ≈ 0.13 seconds per 100 iterations regardless of p or J , because its query cost does not depend on training-set size. GPS-ABC does not: it costs ≈ 0.12 s at $J = 100$ (a 550-point design) but ≈ 0.74 s at $J \in \{10, 50\}$ (a 2,750-point design, five resamples per row; Section 4), a $\sim 6\times$ slowdown that tracks the GP’s $O(n)$ per-query cost against the $5\times$ larger design almost exactly, and the clearest direct demonstration in this paper of the scaling difference Section 1

opens with. Exact ABC-MCMC’s cost explodes with both p and J , since it runs the exact simulator n_s times per iteration. Against exact ABC-MCMC, DNN-ABC is $68\text{--}2381\times$ faster; against GPS-ABC it is a statistical tie at $J = 100$, as expected, since a GP over ~ 550 points is itself cheap to query, but a real and growing advantage at $J \in \{10, 50\}$ once the J -specific design is large enough for the GP’s query cost to matter, the network’s asymptotic query advantage emerging exactly where design size grows, as Section 1 anticipates, though Study II finds that even a real query-cost advantage does not translate into better inference once the surrogate is no longer the binding constraint (Section 5.6). Figure 2 shows that the three posteriors (exact ABC-MCMC, GPS-ABC, DNN-ABC) concentrate on the same true p on a representative data set, confirming that the surrogates reproduce the exact method’s inference and not merely its point estimate.

Table 2: Study I, Table 1: MSE of $\hat{p}$ across five estimators; normalized RMSE $\sqrt{\text{MSE}}/p$ in parentheses. Bold marks the lower of GPS-ABC and DNN-ABC in each row. Δ/SE is the GPS-ABC–DNN-ABC MSE gap in units of its combined Monte Carlo standard error (Section 3.4); values below ~ 2 in magnitude are not clearly resolved at $R = 40$ replicates.

p	J	MOM	MLE	ABC-MCMC	GPS-ABC	DNN-ABC	Δ/SE
10^{-4}	10	9.36e-9 (0.97)	1.02e-8 (1.01)	5.95e-9 (0.77)	7.01e-9 (0.84)	5.25e-9 (0.72)	0.37
10^{-4}	50	8.68e-9 (0.93)	9.59e-9 (0.98)	1.91e-9 (0.44)	1.98e-9 (0.45)	1.77e-9 (0.42)	0.45
10^{-4}	100	4.18e-9 (0.65)	4.64e-9 (0.68)	7.75e-10 (0.28)	7.95e-10 (0.28)	7.68e-10 (0.28)	0.10
10^{-3}	10	1.38e-6 (1.17)	1.60e-6 (1.26)	4.11e-7 (0.64)	6.46e-7 (0.80)	4.58e-7 (0.68)	0.60
10^{-3}	50	3.21e-7 (0.57)	3.85e-7 (0.62)	1.11e-7 (0.33)	1.11e-7 (0.33)	1.33e-7 (0.36)	−0.48
10^{-3}	100	2.71e-7 (0.52)	3.29e-7 (0.57)	4.29e-8 (0.21)	4.48e-8 (0.21)	4.42e-8 (0.21)	0.04
10^{-2}	10	3.42e-5 (0.58)	4.40e-5 (0.66)	1.67e-5 (0.41)	1.77e-5 (0.42)	2.30e-5 (0.48)	−1.02
10^{-2}	50	7.71e-6 (0.28)	1.17e-5 (0.34)	7.01e-6 (0.26)	5.46e-6 (0.23)	9.93e-6 (0.32)	−1.44
10^{-2}	100	4.01e-6 (0.20)	6.08e-6 (0.25)	3.05e-6 (0.17)	3.14e-6 (0.18)	3.37e-6 (0.18)	−0.21

Table 3: Study I, Table 2: mean length of the 95% ABC credible interval for $\hat{p}$. Bold marks the shorter of GPS-ABC and DNN-ABC.

p	J	ABC-MCMC	GPS-ABC	DNN-ABC
10^{-4}	10	2.62e-4	3.11e-4	2.45e-4
10^{-4}	50	1.56e-4	1.50e-4	1.44e-4
10^{-4}	100	1.21e-4	1.16e-4	1.16e-4
10^{-3}	10	2.80e-3	3.32e-3	2.45e-3
10^{-3}	50	1.43e-3	1.66e-3	1.29e-3
10^{-3}	100	9.70e-4	1.20e-3	8.81e-4
10^{-2}	10	1.83e-2	1.96e-2	1.89e-2
10^{-2}	50	1.16e-2	1.74e-2	1.04e-2
10^{-2}	100	7.73e-3	1.25e-2	6.80e-3

4.4 Posterior coverage: what Section 4.2 does not measure

Section 3.2 draws the distinction on which this subsection turns: split-conformal *regression* calibration (Section 4.2, near-exact at 0.9554 on held-out data) is a guarantee about the surrogate’s own predictive interval, not about whether the downstream ABC credible interval for $\hat{p}$ actually brackets the true p 95% of the time. Both Table 2 and Table 3 are silent on the second quantity. We measure it directly by saving the credible interval’s endpoints (not merely its length) for every replicate of every method, exactly as Section 3.4 does for every other comparison in this paper.

A surrogate trained exclusively on $J = 100$ summaries has no way to know that a $J = 10$ observation is intrinsically noisier, so its predictive interval would be calibrated for the wrong noise

level whenever $J \neq 100$: querying the deployed, $J=100$ -only checkpoints of Section 4.1 against these same $J = 10$ replicates, rather than the J -specific surrogates described in Section 4, drops GPS-ABC and DNN-ABC coverage there to 0.43–0.53, roughly half of nominal. Table 5 reports the J -specific surrogates instead, and two patterns are visible in it. First, training each surrogate to know its own culture count recovers most, though not all, of that shortfall: at $J = 10$, GPS-ABC and DNN-ABC now cover at 0.825–0.875, 1–3 Monte Carlo standard errors below the nominal 0.95, a real residual gap, but a small one next to the roughly 0.45 gap the J -blind surrogate leaves, and both reach or exceed nominal by $J = 100$. Neither surrogate is consistently the more under-covering of the two at any J now; Table 3’s interval-length advantage for DNN-ABC is accordingly better evidence of genuine precision at this replicate count

Table 4: Study I, Table 3: wall-clock cost, seconds per 100 MCMC iterations. The final column is the speedup of DNN-ABC over exact ABC-MCMC.

p	J	ABC-MCMC	GPS-ABC	DNN-ABC	Speedup
10^{-4}	10	50.08	0.739	0.130	$386\times$
10^{-4}	50	170.01	0.742	0.127	$1342\times$
10^{-4}	100	301.74	0.126	0.127	$2381\times$
10^{-3}	10	16.33	0.743	0.127	$128\times$
10^{-3}	50	65.32	0.743	0.127	$514\times$
10^{-3}	100	125.75	0.126	0.126	$997\times$
10^{-2}	10	8.52	0.746	0.126	$68\times$
10^{-2}	50	40.07	0.746	0.126	$318\times$
10^{-2}	100	78.39	0.124	0.126	$623\times$

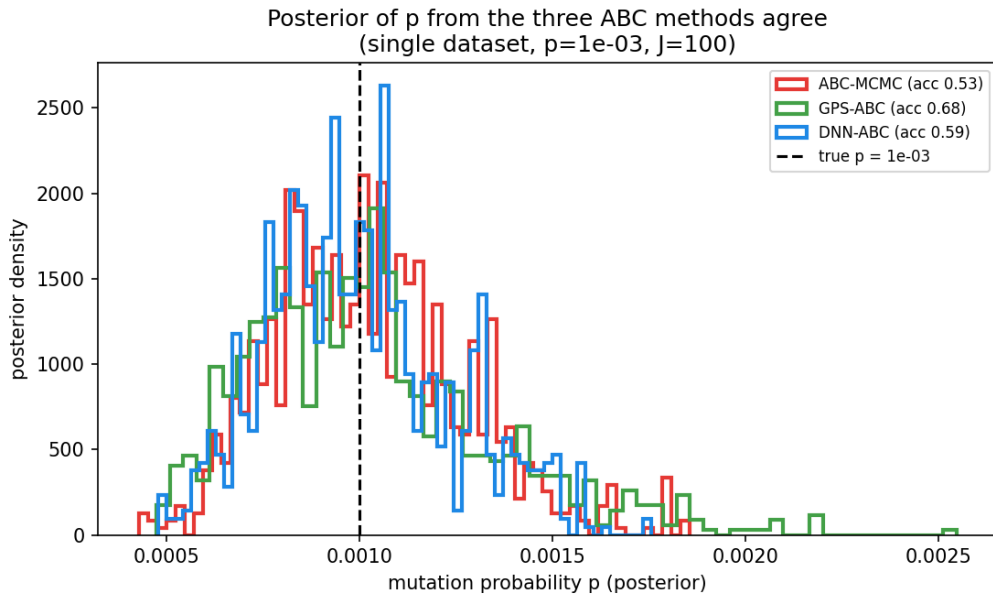


Fig. 2: Study I: ABC-MCMC, GPS-ABC, and DNN-ABC posteriors for θ on a single representative data set.

than it would be if one surrogate’s intervals were also the least calibrated. Second, and separately, exact ABC-MCMC itself is not at nominal at $J = 10$ (0.775–0.850) despite calling the true simulator at the correct J on every iteration, and is not obviously better calibrated there than either J -specific surrogate, a property of the ABC acceptance kernel (the synthetic-likelihood construction of Algorithm 1) at this culture count, not of any surrogate, and not explained by the J -mismatch mechanism above.

The remaining gap at $J = 10$ is worth being precise about rather than explaining away. Split-conformal calibration (Section 4.2) is a marginal guarantee, averaged over the surrogate’s whole training distribution; it does not guarantee the surrogate’s predictive interval is correctly sized at every individual θ , and the $J = 10$ training design is the smallest and noisiest of the three (Section 4), which is the more likely source of the residual gap than any single obvious defect. **The practical lesson is the same regardless of that residual:** a surrogate’s interval-length advantage in Table 3 is

Table 5: Study I: empirical posterior coverage of the 95% ABC credible interval for $\hat{p}$, with binomial Monte Carlo standard error ($\sqrt{\hat{c}(1-\hat{c})/R}$, $R = 40$). This is a different quantity from Table 1’s regression coverage; see Section 4.4.

p	J	ABC-MCMC	GPS-ABC	DNN-ABC
10^{-4}	10	0.775 (0.066)	0.825 (0.060)	0.850 (0.056)
10^{-4}	50	0.900 (0.047)	0.875 (0.052)	0.900 (0.047)
10^{-4}	100	0.925 (0.042)	0.950 (0.034)	0.950 (0.034)
10^{-3}	10	0.800 (0.063)	0.875 (0.052)	0.875 (0.052)
10^{-3}	50	0.975 (0.025)	0.975 (0.025)	0.925 (0.042)
10^{-3}	100	0.950 (0.034)	0.975 (0.025)	0.925 (0.042)
10^{-2}	10	0.850 (0.056)	0.875 (0.052)	0.875 (0.052)
10^{-2}	50	1.000 (0.000)	1.000 (0.000)	0.925 (0.042)
10^{-2}	100	0.950 (0.034)	1.000 (0.000)	0.925 (0.042)

not by itself evidence of equal or better calibration, and training a surrogate that knows the deployment condition it will actually be queried at is necessary, though on this evidence not always fully sufficient, for its ABC posterior to be trustworthy in a way no regression diagnostic computed at the training condition alone can reveal.

5 Simulation Study II: The Two-Stage Mutation Model

Study I holds $a = \delta = 1$ and a single constant mutation probability, as does Lu et al (2023)’s constant-rate benchmark. Their multi-parameter study is commonly read as that same model with more parameters freed, and it is not. It does not vary a or δ around the constant-rate model; it replaces the model outright with a *two-stage* (piecewise-constant) mutation rate,

$$p(t) = \begin{cases} p_1, & 0 < t \leq \tau, \\ p_2, & \tau < t \leq t_p, \end{cases} \quad (2)$$

in which the mutation probability jumps at an unknown time τ . The motivation is experimental: mutagenesis under sub-inhibitory antibiotic stress involves a cell-recovery step before plating, producing two genuinely distinct mutation regimes. Study II therefore estimates (p_1, p_2, τ) jointly. The division rate a is fixed at 1 throughout, as it is in every figure, study and data analysis of Lu et al.

5.1 Ground-truth data

Ground truth is generated by the exact, cell-by-cell simulator, a direct port of the two-stage reference

implementation, over a Latin hypercube of 2,000 design points in $(\log_{10} p_1, \log_{10} p_2, \tau)$ with ten replicates each: 20,000 rows, $J = 100$ cultures per row, $Z_0 = 1$, $a = 1$, $t_p = 10$. A Latin hypercube replaces Study I’s factorial grid because a factorial design wastes points badly in three dimensions. Ranges are $\log_{10} p_1, \log_{10} p_2 \in [-5, -1.3]$ and $\tau \in [0.1, 9.9]$. **The summary statistic is the fourth root.** Lu et al use $\sqrt{X/Z}$ for the constant-rate study of Section 4 (their Figure 1 selects it against $\hat{p}_{\text{MOM}}$ and $\log(Y/Z)$) but switch to $\sqrt[4]{X/Z}$ for the two-stage model, stating that the statistic must be adjusted ”to make the response curve smooth and non-flat” once the mutation rate is piecewise constant; their Algorithm 1 uses the fourth root throughout. Study II therefore takes

$$S = \frac{1}{J} \sum_{i=1}^J (X_i/Z_i)^{1/4},$$

and every surrogate, every baseline and the sampler all consume $\log_{10} S$. Because the ground truth records each culture’s $d_i = \sqrt{X_i/Z_i}$ as well as their mean, S is recoverable exactly as $\sqrt{\overline{d_i^2}}$ without re-simulating anything. The choice is consequential rather than cosmetic, and we can quantify both of its consequences. It lowers the surrogate’s held-out error from $1.19\times$ its irreducible floor to $1.11\times$, and it changes which activation the selection of Section 5.4 returns. A reader reproducing this study under the square root would therefore obtain a different network, not merely a slightly worse one.

This is not Lu et al’s parameter regime. It should be stated plainly, because the difference is large and easy to miss. Their Study II places $\log_{10} p_1, \log_{10} p_2 \in [-11, -7]$ with $\tau \in [0.1, 19.9]$, $t_p = 20$ and a fourth free parameter $\delta \in [0.5, 2]$; ours are $[-5, -1.3]$, $\tau \in [0.1, 9.9]$, $t_p = 10$ and $\delta \equiv 1$. The mutation-rate ranges *do not overlap at all*. The cause is the choice of an exact simulator: their regime is reachable only with the fast approximate sampler of their Algorithm 4, and at $t_p = 10$ any rate below $\approx 10^{-5}$ leaves almost every culture mutant-free. A further consequence is that the constant expected-mutant-count design of Study I (t_p solved so that $E[X] = 20$ at every design point) is abandoned here: with t_p fixed, the expected count ranges from 0.22 to $\approx 1,100$ across the design. What follows is therefore a study of the same *model* in a different regime, not a numerical reproduction of their Table 4, and no such comparison is claimed.

Two further design choices deserve statement. First, t_p is *fixed* at 10 rather than solved per parameter as in Study I. The exact simulator costs $O(e^{at_p})$: at $t_p = 10$ a culture holds $\approx 2.2 \times 10^4$ cells, while at Lu et al’s Study II value of $t_p = 20$ it holds $\approx 5 \times 10^8$, which is not reachable exactly. Their Study II uses the fast approximate simulator throughout for exactly this reason. Second, with t_p fixed, the expected mutant count per culture is $\approx 2.2 \times 10^4 p$, so mutation rates below $\approx 10^{-5}$ leave almost every culture mutant-free and $\bar{d}$ collapses to zero. The stated range keeps every design point informative; no row in the generated data has $\bar{d} = 0$.

The port is validated by a degeneracy argument that is exact rather than statistical: setting $p_1 = p_2$ reduces Eq. (2) to the constant-rate model, and in that case the two-stage simulator reproduces the constant-rate simulator *bit for bit* on a shared random seed. The full 20,000-row dataset was generated in 23.6 minutes of wall time (39,227 core-seconds, 92% parallel efficiency on 30 cores).

5.2 The inference problem is intrinsically asymmetric

Before any method is applied, the ground truth already shows how much the single scalar summary $\bar{d}$ can possibly say about each parameter:

	p_2	p_1	τ
$\text{corr}(\log_{10} S, \cdot)$	0.778	0.337	0.160

p_2 is well determined, p_1 weakly, and τ barely. The asymmetry is intrinsic to the process rather than an artefact of any method applied to it: mutations arising before τ are exponentially rare, because few cells exist that early, so p_1 ’s contribution to the final mutant count is swamped by p_2 ’s. Lu et al report precisely this, describing multi-modal p_1 and τ marginals and attributing them to a lack of identifiability. Any honest evaluation must therefore report per-parameter accuracy rather than a single aggregate that would conceal it, and must prefer error measured on the $\log_{10}$ scale: for a weakly identified parameter the posterior mean sits wherever the prior places its mass, so a normalised natural-scale RMSE diverges without conveying anything.

5.3 The surrogate, and why it is small

The decisive measurement for this study is not a comparison between architectures but the *irreducible noise floor* of the data. Because the held-out target is itself a two-replicate mean, it carries $\mathbb{E}[\sigma^2]/2$ of sampling noise that no model can predict away. On these data $\mathbb{E}[\sigma^2] = 6.90 \times 10^{-4}$, so that floor is $\text{MSE} = 3.45 \times 10^{-4}$. Table 6 reports the architecture ladder against it.

Two conclusions follow, and the second is the more useful of the two. The first is that a network is genuinely required here, and clearly so: the linear control sits at $141\times$ the floor with $R^2 = 0.55$. (Under the square-root target of the earlier draft it sat at $24.8\times$; the fourth root makes the surface harder for a linear model by nearly a factor of six, so the paper’s own choice of statistic strengthens rather than weakens the case for a network.) But capacity is not the binding constraint anywhere above roughly 700 parameters: a 690-parameter network matches a 42,306-parameter one to within 17%, and a 2,402-parameter one to within 8%. LayerNorm and residual depth were likewise statistically indistinguishable, as was the choice among the activations in common use for smooth regression (Section 5.4), though that section also shows the qualifier matters, since several otherwise-reasonable activations outside that group are measurably worse. This contrasts sharply with Study I, where removing BatchNorm changed accuracy by an order of magnitude. On this surface, capacity and depth buy nothing. A study that

Table 6: Study II: accuracy against the irreducible noise floor. MSE is measured on held-out design-point means of $\log_{10} S$; the floor is 3.45×10^{-4} . Three seeds per row.

Hidden layers	Parameters	$\times$ floor	R^2	$\mu\text{s}/\text{query}$
256–128–64	42,306	1.01	0.99681	56
128–64	8,898	1.04	0.99671	42
64–32 (used)	2,402	1.09	0.99656	40
32–16	690	1.17	0.99628	37
16–8	218	1.41	0.99553	38
8–4	78	1.81	0.99426	37
Linear (control)	8	140.88	0.55319	29

reported a ranking among these variants would be reporting seed noise, and reporting it as though it were a finding.

We therefore select the architecture for parsimony rather than accuracy: a two-layer $64 \rightarrow 32$ funnel with GELU/tanh activations (Section 5.4), two output heads, and no BatchNorm. We note explicitly that this is *not* justified by speed. Query latency at this scale is dominated by interpreter and framework call overhead rather than arithmetic, so a $59\times$ reduction in parameter count buys only about 29% less latency (Table 6).

Does the curve-fit tie survive into the downstream task? Table 6 is a curve-fit diagnostic; it does not by itself say whether the same architectures remain tied on the actual ABC-MCMC recovery task, at the replicate count Table 9 uses. We test that directly, mirroring the design applied to the equivalent one-dimensional question (Section 4.1): every candidate in the ladder is first screened at this package’s own quick-check settings (2 replicates), and every candidate survives that screen, so all six are confirmed at Table 9’s own scale (16 replicates, all three truth triples, 3,000 MCMC iterations) with Monte Carlo standard errors on $\text{rmse}_{\log}$ attached to every comparison against the deployed $64 \rightarrow 32$ (`abc/run_experiments_capacity.py`).

The curve-fit tie survives downstream, and it sharpens into something slightly more useful: 32–16 is again the best point estimate, as it is on the one-dimensional model’s own capacity question, but here *no* pairwise comparison against the deployed network is resolved at the 2-MCSE level, not even the largest network tested, 256–128–64 at $17\times$ more parameters than deployed, which is in fact the nominally *worst* performer of the six. This is a stronger form of the conclusion Table 6 already

draws: on this surface, downstream accuracy is flat across more than two orders of magnitude of parameter count in both directions from the deployed size, not merely insensitive to modest shrinkage. We do not redeploy on this evidence alone: the deployed $64 \rightarrow 32$ is what every number in Table 9 and Section 5.6 is reported against, and this analysis cannot show a smaller network is better, only that it is not measurably worse.

Heteroscedasticity. The second output head is not optional here. The within-design-point replicate standard deviation of $\log_{10} \bar{d}$ varies by a factor of 184 across the design and correlates -0.82 with the target itself: where few mutants arise, $\bar{d}$ is small *and* its relative scatter is large. A single homoscedastic noise term, which is what a Gaussian process supplies, cannot represent this, whereas the variance head learns the shape directly. Since the ABC acceptance step consumes that predictive variance directly, an uncalibrated variance corrupts the posterior even when the mean is exact.

5.4 Choosing the activation function

The architecture ladder of Section 5.3 fixes width and depth; activation is a separate choice, tested rather than assumed. We compare ten activations spanning three families (piecewise-linear, smooth saturating, smooth unbounded) on the deployed $64 \rightarrow 32$ network, five seeds each, then the full $10 \times 10 = 100$ cross of layer-wise assignments, screened on validation and confirmed on fifteen fresh seeds to avoid selection optimism. Full methodology and per-pair results are in Supplementary Material S2.

The choice can matter: the best-to-worst spread is 29% of the best row’s MSE, and softplus fails outright, being strictly positive and unable to produce centred activations. But the five leading candidates,

Table 7: Study II: the capacity ladder’s ABC-MCMC recovery, confirmed at Table 9’s own scale (16 replicates, all three truth triples, 3,000 MCMC iterations). Δ/SE is each network’s gap against the deployed 64–32, in combined Monte Carlo standard errors, maximized over the nine parameter-by-truth cells.

Hidden layers	Parameters	Mean $\text{rmse}_{\log}$	Resolved vs. deployed
GP (GPS-ABC baseline)	—	0.990	—
32–16	690	1.067	0/9 ($\max \Delta/\text{SE} = 1.63$)
128–64	8,898	1.105	0/9 ($\max \Delta/\text{SE} = 1.16$)
16–8	218	1.133	0/9 ($\max \Delta/\text{SE} = 1.06$)
64–32 (deployed)	2,402	1.142	—
8–4	78	1.149	0/9 ($\max \Delta/\text{SE} = 1.32$)
256–128–64	42,306	1.208	0/9 ($\max \Delta/\text{SE} = 1.55$)

GELU, tanh, LeakyReLU, ReLU and Mish, are mutually indistinguishable. Selection optimism is real and not small: finalists perform on average 45% of the confirmation table’s spread worse on fresh seeds than on the seeds that chose them, which is why evaluation never reuses a selection seed. GELU into tanh is the best point estimate and the pair we deploy, with held-out test MSE $1.06\times$ the irreducible floor; both activations are smooth and free of the dead-unit failure mode, so the deployed network is everywhere differentiable. No conclusion in this paper turns on the choice: the downstream comparison of Section 5.6 is unchanged in kind under any of the leading pairs.

5.5 Surrogate fit and calibration

The surrogate is trained on replicates 1–5, early-stopped and conformally calibrated on replicates 6–8, and evaluated on replicates 9–10, so every design point appears in every split and no parameter combination leaks across them. Because the splits contribute different numbers of replicates per design point, each is scored against *its own* floor ($\mathbb{E}[\sigma^2]/r$ for $r = 5, 3, 2$).

The surrogate sits at its noise floor on all three splits, and the train and test ratios (1.16 and 1.05) are close enough that overfitting is not detectable: with 2,402 parameters against 10,000 training rows, and early stopping on a held-out split, there is little room for it. The conformal scale factor of 1.019 is a further diagnostic in its own right: the variance head was already almost calibrated before the correction was applied.

5.6 ABC inference

Inference is a random-walk Metropolis–Hastings sampler over the full vector $\theta = (\log_{10} p_1, \log_{10} p_2, \tau)$, with component-wise truncated proposals and Hastings corrections for the truncation. This is a genuine joint three-parameter posterior, not a scalar inference with covariates.

ABC-MCMC, GPS-ABC and DNN-ABC share the sampler and differ only in how the summary statistic and its uncertainty are obtained at each proposal, exactly as in Study I. The constant-rate MOM and MLE estimators of Study I have no counterpart here: they assume a single mutation rate and cannot identify p_1 , p_2 or τ individually, so Study II compares the surrogates against each other: the neural one, and two Gaussian processes (ours and a faithful reproduction of the reference implementation).

Two Gaussian-process baselines, and why both are reported. Our GP baseline is stronger than the one Lu et al actually use, and reporting only ours would overstate what the published method achieves. Theirs, in the reference implementation for this study, fits an *isotropic* squared exponential to the untransformed rates (p_1, p_2, τ) predicting the untransformed statistic, with a single noise term. Ours uses an anisotropic kernel with one length scale per input, log-scaled inputs, and a learned white-noise term.

The gap those choices make is large. On raw-scale surrogate fit the reference configuration reaches $R^2 \approx 0.71$, while an otherwise identical anisotropic kernel on the same 300 design points reaches ≈ 0.99 : a single shared length scale cannot serve inputs whose ranges differ by a factor of

Table 8: Study II: surrogate fit. Each split is compared to its own irreducible floor. Coverage is of the nominal-95% predictive interval after a split-conformal scale factor of 1.019.

Split	n	MSE (design means)	$\times$ its own floor	95% coverage
Train (reps 1–5)	10,000	1.60×10^{-4}	1.16	0.951
Validation (6–8)	6,000	2.95×10^{-4}	1.28	0.950
Test (9–10)	4,000	3.63×10^{-4}	1.05	0.955

roughly 200 (p_1, p_2 span ≈ 0.05 , τ spans ≈ 9.8). We therefore report both, fit from the same rows under the same budget so that the comparison isolates the model rather than the data. Note that the reference performs ABC on the untransformed statistic, as its own sampler does, whereas our pipeline works in $\log_{10}$ throughout; the observation is placed on whichever scale the surrogate reports.

What NPE is, and how it differs from the three methods above. Table 9 adds a fourth method, an amortized neural posterior estimator (NPE). Where GPS-ABC and DNN-ABC learn a surrogate for the *simulator* and then plug it into the same ABC-MCMC sampler exact ABC-MCMC uses, NPE instead learns the *posterior* itself, directly, as a conditional density $p(\theta \mid S)$, with no sampler in the loop at all. Concretely, we train a mixture-density-network density estimator (sbi’s single-round NPE-C, Tejero-Cantero et al, 2020) once, offline, on the same 16,000 training rows DNN-ABC trains on, conditioned on the same fourth-root summary statistic every other method here is scored against. Training happens exactly once, before any observed data is seen; at inference time, the trained estimator is queried a single time at the observed S_{obs} and returns posterior samples directly: no MCMC chain, no acceptance step, no ϵ -tolerance, and no per-observation re-simulation. The only thing NPE shares with GPS-ABC and DNN-ABC is the training budget; the inference procedure is a different paradigm, not a fourth variant of the same one, which is the comparison this subsection is built to make precise rather than assume.

Why it is tested here and not in Study I. We test it here rather than in Study I for a reason specific to this model’s structure, not a matter of convenience: Study I has a single parameter that is well identified by construction, so it cannot pose the question that makes an amortized-vs-surrogate comparison informative in the first place: whether the two approaches behave differently *as a function*

of how identifiable a parameter is. Study II can, because Section 5.2 already establishes that p_2 is well identified by the summary statistic while p_1 and τ are not, within the same jointly estimated posterior. That contrast, not the choice between ABC-MCMC and NPE in the abstract, is what this comparison is actually testing, and Study II is the one place in this paper where it can be tested at all. It is also, not incidentally, the model that is this paper’s own extension of Lu et al rather than a direct reproduction of it (Section 5), so it is where we test questions beyond the surrogate-vs-surrogate comparison Study I is built around; the architecture-family comparison of Section 5.7 is the other one.

Table 9 reports parameter recovery over three truth triples with sixteen replicates each, 3,000 MCMC iterations and 1,000 burn-in. Errors are RMSE on the $\log_{10}$ scale for p_1, p_2 and in absolute time units for τ , for the reason given in Section 5.2.

Two things in this table matter, and neither favours the neural surrogate. We set them out in turn.

The two surrogates are statistically tied. Attaching Monte Carlo standard errors to every cell and comparing DNN-ABC against the strengthened GPS-ABC at the 2-MCSE level, seven of the nine parameter-by-truth comparisons are ties and the remaining two favour GPS-ABC; none favours the network. The two are p_2 at $(1 \times 10^{-4}, 1 \times 10^{-2}, 3)$, difference $+0.050 \pm 0.039$, and τ at $(2 \times 10^{-3}, 8 \times 10^{-3}, 5)$, $+0.411 \pm 0.409$, both marginal. We state this plainly because Study I found a consistent DNN advantage, and it would be natural to assume the advantage carries over. It does not. The reason is already visible in Section 5.3: when both surrogates fit the response surface essentially perfectly relative to its own noise, the surrogate has stopped being the bottleneck, and downstream accuracy is set by the identifiability of the model rather than by the quality of the emulator. A surrogate can only help where surrogate error is what limits the

Table 9: Study II: joint recovery of (p_1, p_2, τ) , averaged over three truth triples, $J = 100$, 16 replicates per cell. Errors are $\log_{10}$ -scale RMSE for p_1, p_2 (so 1.0 means an order of magnitude) and absolute for τ . Two Gaussian-process baselines are reported: the strengthened one used throughout, and one built to match the reference implementation of Lu et al (isotropic kernel, untransformed inputs and target). Exact ABC-MCMC (the same synthetic-likelihood sampler Study I uses, calling the true simulator $n_s = 4$ times per iteration) is included as the expensive gold-standard baseline. NPE has no acceptance step, so its rate column reads “—”.

Method	Parameter	RMSE	Mean 95% CI width	Coverage	Accept. rate
ABC-MCMC (exact)	p_1	1.416	3.52e-2	1.000	0.135
	p_2	0.231	2.76e-2	0.938	
	τ	1.772	8.76	0.979	
GPS-ABC (strengthened)	p_1	1.391	3.46e-2	1.000	0.371
	p_2	0.156	3.22e-2	1.000	
	τ	1.422	9.07	1.000	
GPS-ABC (reference)	p_1	1.309	3.47e-2	1.000	0.810
	p_2	0.241	3.05e-2	1.000	
	τ	1.799	9.01	1.000	
DNN-ABC	p_1	1.407	3.43e-2	0.938	0.143
	p_2	0.258	3.07e-2	0.979	
	τ	1.753	8.64	1.000	
NPE	p_1	1.373	3.10e-2	1.000	—
	p_2	0.089	2.89e-2	1.000	
	τ	1.425	8.77	1.000	

answer. That condition held in Study I and does not hold here.

DNN-ABC and GPS-ABC are not resolvably different on coverage either. Coverage of the nominal 95% credible interval is 0.938–1.000 for DNN-ABC and 1.000 throughout for both Gaussian processes. With sixteen replicates per cell, coverage moves in steps of $1/16 = 0.0625$, so DNN-ABC’s 0.938 for p_1 is fifteen replicates out of sixteen against the GP’s sixteen, a difference of one, well inside the per-cell Monte Carlo standard error (coverage MCSE ± 0.08 at this replicate count). We therefore do not claim a calibration difference between the methods. Mean interval widths are likewise close (p_1 : 3.43×10^{-2} for DNN-ABC against 3.46×10^{-2} for GPS-ABC).

What the coverage column does show is that every surrogate over-covers on τ , at 1.000 across the board, and NPE does too despite having no acceptance kernel at all. That is what a nearly unidentified parameter looks like rather than a defect of any surrogate: the credible interval spans most of the prior box, so it contains the truth

essentially by construction. Note also that this is not a failure of the surrogate’s own predictive calibration, which is near-exact on held-out data (Table 8, coverage 0.955); it is a property of the ABC acceptance kernel built on top of it. Exact ABC-MCMC’s own τ coverage (0.979) sits close to but not exactly at that ceiling (the one cell in the whole table where it does not round to 1.000) and its p_2 coverage (0.938) lands almost exactly on the nominal rate, better calibrated there than every surrogate and NPE alike, all four of which over-cover p_2 at 1.000. The exact sampler’s own pilot-simulation noise, rather than a fitted surrogate’s or an amortized estimator’s smoother predictive variance, is the most likely source of that difference, though at sixteen replicates per cell it is not itself resolved from the ties above.

Does the tolerance ϵ drive this, rather than identifiability? That claim is tested here rather than asserted. We re-ran GPS-ABC, GPS-ABC-ref and DNN-ABC (NPE has no acceptance kernel and no ϵ , so it is excluded by construction) across a $16 \times$ range of ϵ around the paper’s fixed value, 0.0025

to 0.04, on the same observations. Both Gaussian processes are essentially untouched: coverage sits at 1.000 throughout the entire range for all three parameters, with one single-cell exception (p_1 at the two extremes, 0.979), and mean interval width moves by at most a few percent. DNN-ABC is more ϵ -sensitive: coverage on p_1 and p_2 rises from 0.875 at the tightest ϵ to 1.000 at the loosest, a change resolved at the 2-MCSE level ($\Delta/\text{SE} = 2.62$), but even its lowest measured coverage, at half the paper’s default tolerance, is not below nominal by more than this replicate count’s own noise. Widening or narrowing ϵ by an order of magnitude in either direction therefore does not reproduce or remove the over-coverage on τ that motivates this question, which is the strongest single piece of evidence in this paper that the cause is the model’s identifiability rather than a tunable property of the kernel.

NPE is the one method that does favour an amortized estimator here, and only on the well-identified parameter. On p_2 , NPE’s RMSE (0.089) beats every surrogate at every truth tested, never the reverse, and the gap is resolved at the 2-MCSE level in seven of the nine surrogate comparisons (all three against the reference GP, two of three each against the strengthened GP and DNN-ABC; the remaining two are ties, never a surrogate win). On p_1 and τ , the weakly identified parameters, NPE is a mix of ties and occasional wins and losses against each GP variant, consistent with Section 5.2’s point that no method can extract information the summary statistic does not carry: an amortized estimator trained directly on simulator draws is not exempt from that limit, and Table 9 does not show it beating the identifiability ceiling either. The asymmetry is the finding: where the model actually identifies a parameter, learning the posterior directly from simulations outperformed substituting a surrogate into the same ABC-MCMC kernel Study I uses, resolved beyond replicate noise; where it does not, no method, amortized or not, does better than any other. Training cost and per-dataset sampling cost are negligible here for the same reason as Study I (seconds to train, milliseconds per dataset); the finding is about accuracy, not speed, and is the more interesting for it.

The same asymmetry holds against the exact simulator itself, not just the surrogates. Table 9’s ABC-MCMC row is not a cross-study comparison read off Table 2; it is the same synthetic-likelihood

sampler run on Study II’s own three truth triples, so NPE’s recovery can be validated against the same gold-standard baseline Study I uses throughout, not only against the known ground truth. Across all nine parameter-by-truth comparisons, NPE never loses to exact ABC-MCMC and wins three at the 2-MCSE level: p_2 at two of the three truths ($t = 5.39$ and $t = 3.32$) and τ at the third ($t = 5.55$), with the remaining six ties. This is the paper’s most direct evidence for the asymmetry above: an amortized estimator trained once on simulator draws matches or beats not merely a cheap surrogate but the expensive exact sampler it is meant to substitute for, exactly where the model actually identifies a parameter, and never underperforms it anywhere in this table.

Does this pattern hold at Study I’s replicate count? Table 9 uses sixteen replicates per cell; Study I uses forty. We reran the four methods that do not call the exact simulator inside their sampler (GPS-ABC, GPS-ABC-ref, DNN-ABC and NPE, all cheap regardless of replicate count since none re-simulates per MCMC iteration) at $R = 40$, leaving exact ABC-MCMC at $R = 16$ (its validation above already cost several hours; $2.5\times$ that is not a proportionate use of compute for a confirmatory check). Table 10 reports it.

Every point estimate is within a few percent of its $R = 16$ counterpart, and the two findings this section builds on both survive intact, more sharply resolved than before rather than less: DNN-ABC still never beats GPS-ABC at the 2-MCSE level (four of nine comparisons now resolved, up from two, all favouring GPS-ABC), and NPE still never loses to any surrogate on p_2 (now resolved in eight of nine surrogate comparisons, up from seven). Tripling the replicate count sharpens the same picture rather than changing it, which is what this check was for.

We record one diagnostic that qualifies the comparison: DNN-ABC’s acceptance rate is 0.143 against the strengthened GP’s 0.371 and the reference GP’s 0.810, so the three chains mix very differently at the shared proposal scale and some of the DNN’s disadvantage may be tuning rather than method. The reference GP’s high acceptance is itself informative: a flatter, less discriminating surrogate accepts more freely. Per-method proposal tuning is the obvious next step and is not attempted here.

Table 10: Study II at $R = 40$, matching Study I’s replicate count: same four methods, same three truth triples, same 16,000-row training budget. Compare against Table 9’s $R = 16$ figures directly: point estimates move by noise, not by any qualitative change.

Method	Parameter	RMSE	Mean 95% CI width	Coverage	Accept. rate
GPS-ABC (strengthened)	p_1	1.400	3.57e-2	0.983	0.372
	p_2	0.160	3.22e-2	1.000	
	τ	1.524	9.07	1.000	
GPS-ABC (reference)	p_1	1.311	3.45e-2	1.000	0.808
	p_2	0.236	3.04e-2	1.000	
	τ	1.807	9.02	1.000	
DNN-ABC	p_1	1.416	3.44e-2	0.925	0.143
	p_2	0.285	2.95e-2	0.950	
	τ	1.869	8.65	0.983	
NPE	p_1	1.374	3.11e-2	1.000	—
	p_2	0.088	2.93e-2	1.000	
	τ	1.416	8.78	1.000	

Taken together, Study II is best read as a negative-to-neutral result on the surrogate question and a positive one on the modelling question. The neural surrogate matches, but does not beat, a Gaussian process on downstream inference in this regime, because the regime is one where neither is the limiting factor. What does bind is the model’s intrinsic identifiability: τ carries an RMSE of roughly 1.7–1.9 time units against a prior range of 9.8, and p_1 is recovered no better than an order of magnitude by either method.

5.7 Does architecture matter here? A structural comparison

The heteroscedastic MLP of Section 5.3 treats its three inputs ($\log_{10} p_1, \log_{10} p_2, \tau$) as an unordered feature vector. Whether an architecture built around a different inductive bias, a convolution or a recurrence, would do better is a natural question, and one we test directly rather than assume. Radev et al (2020)’s alignment principle predicts a plain feedforward network should be at least as good here, since our three inputs have neither temporal order nor exchangeability. We compare a 1-D CNN, a GRU, and an LSTM against the deployed MLP on both surrogate fit quality and the full ABC-MCMC recovery task, same truth triples, replicates, and iterations as the main comparison. Full setup and per-family tables are in Supplementary Material S3; code and trained checkpoints for

all three architecture families are in the repository (Section 7).

CNN1D statistically ties the deployed FFN on fit quality at $3.6\times$ its query cost; LSTM and RNN are measurably worse (+12.6%, +20.6% MSE), consistent with the structural prediction. Across 54 pairwise ABC-recovery comparisons at the 2-MCSE level, 91% are ties, and *none* favours a new architecture family anywhere. Interval calibration adds nothing to the picture either way: pooled coverage is 0.951–0.979 for the four neural families against a nominal 0.95, a spread under three percentage points that supports no calibration conclusion. Neither a spatial prior (CNN1D) nor a temporal one (RNN, LSTM) improves on the architecture-agnostic FFN for this input, consistent with Radev et al (2020)’s prediction, and the deployed 3-D surrogate throughout this paper remains the plain heteroscedastic MLP of Section 5.3.

5.8 Scope and what remains open

Four limitations should be read alongside the results above. The first two concern the regime this study occupies, the third the model it uses, and the fourth an error we made and corrected.

The plating time is not the paper’s. $t_p = 10$ here against Lu et al’s $t_p = 20$, forced by the exponential cost of exact simulation. Reaching their regime requires the fast two-stage simulator; it

is implemented but not yet validated against the exact simulator at that scale, so we do not report results in that regime and no direct numerical comparison with their Table 4 is claimed.

The parameter regime is not the paper’s either, and the gap is wider than the plating time alone. Lu et al’s Study II places $\log_{10} p_1, \log_{10} p_2 \in [-11, -7]$; ours span $[-5, -1.3]$. The two ranges do not overlap. The cause is the same choice of an exact simulator: at $t_p = 10$ a rate below $\approx 10^{-5}$ leaves almost every culture mutant-free. A consequence is that Study I’s design rule (t_p solved so that $E[X] = 20$ at every design point, which we do follow in Section 4) cannot be maintained here, and the expected mutant count instead ranges from 0.22 to roughly 1,100 across the design. What we report is the same model studied in a different and easier regime.

The model has no δ . The reference two-stage implementation carries no differential mutant growth rate, making this a genuine three-parameter study; Lu et al’s full specification is four-dimensional. Our simulator threads δ through so the extension does not require a rewrite.

A local copy of the reference implementation had drifted from the published code, and a prior version of this study reported a spurious finding as a result. We previously reported that the reference implementation’s mutation-time indexing was ambiguous between two conventions – a cell’s own division time versus its parent’s – and that an earlier version of this study had trained surrogates under one convention while drawing observed data under the other. Re-checking the local reference file against Lu et al’s own public repository (https://github.com/lruijin/ABC_mutation-rate) shows that file did not match the published source at all: it used a different function signature and had introduced a parent/offspring branch that does not exist in the real code, which we checked in full, including every subfolder. The published algorithm indexes $p(t)$ by each cell’s own division time unconditionally, with no alternative convention anywhere in the source. There was accordingly no genuine ambiguity to resolve and no cross-convention mismatch in the reported pipeline: its default had always matched the one convention the published algorithm implements, so no reported quantity in this paper changes. We correct the record here rather than silently drop the earlier claim, since

an earlier draft of this section reported it as a positive finding and it should not stand uncorrected. The local reference copy has been replaced with a verified-identical copy of the published file, and the corresponding branch in the simulator and its test have been removed.

6 Discussion

The two studies answer the question of Section 1 with a condition rather than a verdict, and that condition is the contribution of this paper we would most emphasize. Across both studies, the neural surrogate’s advantage over the Gaussian process appears exactly where theory predicts it should: in surrogate quality and cost scaling as the useful training-set size grows (Section 4.1), and it compounds when the exact simulator used to build the training design is itself expensive, as it is here. In the one-dimensional case, where a Gaussian process fit on ~ 800 points is already close to a well-specified problem’s optimum, the two surrogates are close to interchangeable on point accuracy and query speed, and DNN-ABC’s advantage is concentrated in tighter, still-calibrated intervals. Study II sharpens the caveat considerably: there the surrogate-quality advantage vanishes entirely, because both surrogates reach the data’s noise floor (Table 6), and advantages in emulator quality do not automatically translate into a better *downstream ABC posterior*: point accuracy is a near-tie, and every method over-covers its credible intervals, at 1.000 throughout for τ , despite held-out predictive calibration that a plain regression diagnostic reports as near-exact (Table 8). The over-coverage is a property of the ABC acceptance kernel built on top of the surrogate rather than of the surrogate’s own predictions.

A related finding, and one we would place beside the surrogate-quality result above rather than after it, comes from Section 5.6’s amortized-estimator comparison. The identifiability ceiling that stops a better surrogate from helping in Study II turns out not to be specific to the surrogate-inside-ABC-MCMC design this paper otherwise tests throughout: an NPE trained directly on simulator draws, with no surrogate step and no acceptance kernel in between, shows the identical asymmetry. It beats every surrogate tested, and the exact simulator-based sampler itself, resolved beyond replicate noise in most comparisons, on p_2 ,

the one parameter the model identifies well, and it does no better than any other method, including exact ABC-MCMC, on p_1 or τ , the two the model does not. A single estimator showing that pattern could be a quirk of the estimator; two structurally unrelated ones (a surrogate substituted into an MCMC kernel, and a density estimator trained once on simulation draws with no kernel at all) agreeing on exactly where the ceiling sits, against both the cheap surrogates and the expensive exact sampler alike, is evidence that the ceiling belongs to the model rather than to either method.

We take this to be a broader methodological point for anyone deploying a conformally calibrated regression surrogate inside a likelihood-free sampler, and one that is easily overlooked. Split-conformal calibration (Lei et al, 2018) is a strong, distribution-free guarantee, and Sections 4.2 and 5.5 show it holding to high precision in both studies. But it is a guarantee about the regression problem the surrogate was trained to solve, not about the coverage of whatever downstream quantity the surrogate’s uncertainty is subsequently plugged into. The guarantee does not automatically extend from one to the other, and the two can come apart; here they do.

A second finding is purely architectural, recurs across both studies, and is worth stating plainly for anyone building a similar surrogate. The single largest driver of surrogate quality in the one-dimensional architecture search (Section 4.1, Supplementary Table S1) was *not* the choice of activation function, which was a near-tie among several reasonable candidates. It was the presence or absence of BatchNorm (Ioffe and Szegedy, 2015). Study II qualifies even this: on that surface no architectural choice mattered at all, because every candidate already sat at the noise floor (Table 6), which is itself worth measuring before any architecture search is run. An otherwise-standard network with BatchNorm fit the one-dimensional curve eleven times worse than the identical network without it. On a smooth regression problem with a bounded, one-sided input domain, BatchNorm’s per-minibatch statistics appear to be actively counter-productive; LayerNorm (Ba et al, 2016), which normalizes per-sample, avoided this failure mode entirely in the deeper three-dimensional network.

A third finding, from the architecture-family comparison of Section 5.7, is best read as a companion to this one rather than a separate result: BatchNorm was a case of an architectural component actively fighting the problem’s structure, whereas CNN1D/RNN/LSTM on the two-stage inputs are a case of architectural components with *nothing to find*: there is no spatial or temporal pattern in three unordered physical parameters for a kernel or a recurrence to exploit, so the honest expectation, confirmed by Radev et al (2020)’s stated design principle and by our own result, is indifference-to-mild-harm rather than either a large gain or a BatchNorm-scale failure. The practical lesson we would draw for anyone designing a similar surrogate is to let the symmetry of the input, not habit or a desire for architectural novelty, choose the architecture family before any search over its hyperparameters begins.

Scope and limitations. This paper is a simulation study evaluated entirely under known ground truth; we do not apply the method to a real fluctuation-experiment data set, as Lu et al (2023) do for their own estimator in their real-data application to drug-resistance experiments. We view a real-data application, ideally to the same or a comparable fluctuation-experiment data set, as the natural and immediate next step, and we are explicit that the comparisons in this paper are not a substitute for one: a simulation study can establish that an estimator is well calibrated and competitive under a correctly specified model, but only a real-data application can speak to robustness under the model misspecification that any laboratory data set will exhibit to some degree. The one-dimensional study uses 40 replicates per cell (versus Lu et al’s 100); Section 3.4 reports Monte Carlo standard errors throughout for exactly this reason, and Sections 4.3 and 5.6 use them to identify which point-accuracy comparisons are, and are not, resolved at this replicate count, rather than over-interpreting numerically lower MSE as a real effect in every cell. The three-dimensional study’s three truth triples at a single culture count, while chosen to span the parameter space broadly, do not cover it exhaustively. The exact ABC-MCMC prior is bounded in both studies to keep the exact simulator feasible inside the sampling loop; surrogates are trained on a wider range and only queried within it. Finally, Study II’s interval miscalibration is, by construction, a property of the

acceptance-kernel construction used here rather than of the surrogates’ own predictive calibration, which is near-exact on held-out data; whether a kernel-side correction restores nominal coverage for both surrogates simultaneously is an open question.

7 Reproducibility

Code and data availability. All code, data, trained model weights, and the exact configuration used for every table and figure in this paper are available at <https://github.com/sleetch1/DNN-ABC-mutation-rates> under a public repository, with no access restrictions. The one-dimensional ground-truth data set (Section 4) is committed directly to the repository as a plain-text, non-proprietary CSV file; the two-stage ground-truth data set (Section 5.1) is likewise committed in the same format, together with the exact simulator that generated it and the reference implementation it was ported from, so that it can be independently re-run rather than taken on faith. Every CSV’s columns are self-documenting from their header row (parameter values, replicate index, and the resulting summary statistic or estimator output); no external data dictionary is required to interpret them.

Computational environment. All code is Python (minimum versions pinned in the repository’s `requirements.txt`: `numpy`, `pandas`, `scipy`, `matplotlib`, `scikit-learn` for the Gaussian-process baseline, and `torch` for the neural surrogates). No GPU is required for any part of the pipeline; every model in both studies trains on CPU in seconds to minutes.

Instructions for reproduction. Each study ships a single entry point, `run_all.py`, that retrains its surrogate, reproduces its ABC tables, recomputes the Monte Carlo standard errors, and regenerates every figure. It takes no command-line arguments: it asks at the console whether to run a short pipeline check or the full reported settings, so the study can be reproduced from an editor as well as a terminal, and it runs under whichever Python interpreter invokes it. The underlying steps remain individually runnable and are documented in each package’s guide, in the order the results are presented in this paper: data generation, the two rounds of architecture search, the activation selection, training and conformal calibration, the ABC

tables, Monte Carlo standard errors, and figure generation.

Study II’s default run trains NPE inline (seconds) alongside the three surrogate-inside-MCMC methods, so Table 9’s NPE column needs no separate step. Its exact ABC-MCMC row is the one part of this paper’s results `run_all.py` does not reproduce by default, because it is genuinely expensive (several hours; `Models/3D/HOW_TO_RUN.md` has the measured figures) rather than merely slow to wire up: `abc/add_exact.py` backfills it onto an existing run without re-simulating the other methods’ observations, and `run_all.py`’s third menu option reruns everything from scratch including it. The ϵ -tolerance sweep of Section 5.6 (`abc/eps_sweep.py`) and the $R = 40$ confirmatory check of Table 10 (`run_experiments.py --reps 40 --no-sim`) are each a single additional command, cheap for the same reason the exact baseline is not: neither calls the true simulator inside the sampling loop.

The two-stage package also ships a validation script that re-derives, rather than asserts, a fact the analysis depends on: that the two-stage simulator reduces to the constant-rate one in both degenerate limits, checked in distribution against an independently ported constant-rate implementation (the two draw their random numbers in a different order per generation, so agreement is checked by a two-sample test on matched moments rather than by a shared-seed replay). The summary statistic is likewise recomputed from the per-culture values stored in that file rather than from a cached aggregate, so the choice of root in Section 5.1 is a single switch and cannot drift between the surrogate, the baselines and the sampler. Every number reported in Tables 2–9 and in the architecture-family comparison of Section 5.7 (Supplementary Tables S5–S6) is produced by these scripts directly from the committed data, with no manual post-processing; the Monte Carlo standard errors and Δ/SE columns introduced in Section 3.4 are computed from the same per-replicate output each script already writes to disk.

8 Conclusion

A heteroscedastic neural-network surrogate, calibrated by split-conformal prediction and substituted directly for the Gaussian process inside Lu et al’s ABC-MCMC sampler, matches or improves

on GPS-ABC in a direct one-dimensional reproduction of their benchmark. Extending the same construction to the two-stage model (p_1, p_2, τ) , however, does *not* reproduce that advantage: there the neural and Gaussian-process surrogates are statistically tied in seven of nine comparisons with the other two favouring the Gaussian process, and we report the result rather than tune it away. The explanation we offer for that failure is general, and it is the more useful contribution of Study II. A surrogate can only improve an inference where surrogate error is what limits it. Once the emulator sits at the data’s irreducible noise floor (as every network we tried does, across three orders of magnitude of model size), the binding constraint becomes the identifiability of the model itself, and no better emulator can reach it. The practical consequence is a cheap diagnostic that we would encourage as routine: measure the noise floor before comparing architectures, because it determines whether the comparison can report anything at all.

Section 5.6’s amortized comparison reinforces this reading rather than complicating it. An NPE trained directly on simulation draws, bypassing the surrogate-inside-MCMC design entirely, shows the identical asymmetry: it beats every surrogate, and the exact simulator-based sampler itself, on p_2 , the one well-identified parameter, and does no better than any of them on p_1 or τ , the two that are not. That two structurally different estimators (one a surrogate substituted into an existing sampler, the other trained once on simulation draws with no sampler at all) agree on exactly where the ceiling sits is stronger evidence than either alone that the ceiling belongs to the model, not the method, and it is this joint result, on the two-stage model that is Lu et al’s own multi-parameter contribution, that we take to be this paper’s central finding. Both the surrogate-substitution framework and the diagnosis of where its guarantees do and do not transfer downstream should apply beyond this particular mutation-rate model to other ABC procedures built on a regression surrogate for an expensive simulator.

Declarations

Data availability. The ground-truth data sets analysed in this study (Section 4 for the one-dimensional constant-rate model and Section 5.1

for the two-stage model) are committed as plain-text CSV files in the code repository accompanying this paper, <https://github.com/sleetch1/DNN-ABC-mutation-rates>. [PLACEHOLDER: insert the specific release tag or archived-version DOI corresponding to this submission once one has been minted.]

Code availability. All code used to produce the results in this paper (surrogate training, ABC-MCMC sampling, Monte Carlo standard error computation, and figure/table generation) is available in the same repository as above, <https://github.com/sleetch1/DNN-ABC-mutation-rates>. A single entry point per study, `run_all.py` (see Section 7), reproduces the full pipeline in one command. [PLACEHOLDER: insert the release tag or archived-version DOI corresponding to this submission once one has been minted.]

Competing interests. [PLACEHOLDER: the authors will state any competing interests here prior to submission.]

Funding. [PLACEHOLDER: the authors will state funding sources, if any, here prior to submission.]

Author contributions. [DRAFT, please confirm or edit before submission: S.L. conceived and implemented the surrogate methods, ran all simulation studies and analyses, and wrote the manuscript. H.Z. and X.W. supervised the research and critically reviewed the manuscript. All authors read and approved the final manuscript.]

References

- Ba JL, Kiros JR, Hinton GE (2016) Layer normalization. arXiv preprint arXiv:160706450
- Beaumont MA, Zhang W, Balding DJ (2002) Approximate Bayesian computation in population genetics. *Genetics* 162(4):2025–2035. <https://doi.org/10.1093/genetics/162.4.2025>
- Cranmer K, Brehmer J, Louppe G (2020) The frontier of simulation-based inference. *Proceedings of the National Academy of Sciences* 117(48):30055–30062. <https://doi.org/10.1073/pnas.1912789117>

- Elfving S, Uchibe E, Doya K (2018) Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks* 107:3–11. <https://doi.org/10.1016/j.neunet.2017.12.012>
- Fan Y, White S (2026) Neural posterior estimation on exponential random graph models: evaluating bias and implementation challenges. *Statistics and Computing* 36. <https://doi.org/10.1007/s11222-026-10896-8>
- Greenberg DS, Nonnenmacher M, Macke JH (2019) Automatic posterior transformation for likelihood-free inference. In: *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pp 2404–2414
- Gutmann MU, Corander J (2016) Bayesian optimization for likelihood-free inference of simulator-based statistical models. *Journal of Machine Learning Research* 17(125):1–47
- Hendrycks D, Gimpel K (2016) Gaussian error linear units (GELUs). *arXiv preprint arXiv:160608415*
- Hermans J, Delaunoy A, Rozet F, et al (2022) A crisis in simulation-based inference? beware, your posterior approximations can be unfaithful. *Transactions on Machine Learning Research*
- Ioffe S, Szegedy C (2015) Batch normalization: Accelerating deep network training by reducing internal covariate shift. In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pp 448–456
- Järvenpää M, Corander J (2023) On predictive inference for intractable models via approximate Bayesian computation. *Statistics and Computing* 33(2):42. <https://doi.org/10.1007/s11222-022-10163-6>
- Järvenpää M, Gutmann MU, Vehtari A, et al (2021) Parallel Gaussian process surrogate Bayesian inference with noisy likelihood evaluations. *Bayesian Analysis* 16(1):147–178. <https://doi.org/10.1214/20-BA1200>
- Lei J, G’Sell M, Rinaldo A, et al (2018) Distribution-free predictive inference for regression. *Journal of the American Statistical Association* 113(523):1094–1111. <https://doi.org/10.1080/01621459.2017.1307116>
- Lu R, Zhu H, Wu X (2023) Estimating mutation rates in a Markov branching process using approximate Bayesian computation. *Journal of Theoretical Biology* 565:111467. <https://doi.org/10.1016/j.jtbi.2023.111467>
- Marin JM, Pudlo P, Robert CP, et al (2012) Approximate Bayesian computational methods. *Statistics and Computing* 22(6):1167–1180. <http://doi.org/10.1007/s11222-011-9288-2>
- Marjoram P, Molitor J, Plagnol V, et al (2003) Markov chain Monte Carlo without likelihoods. *Proceedings of the National Academy of Sciences* 100(26):15324–15328. <https://doi.org/10.1073/pnas.0306899100>
- Meeds E, Welling M (2014) GPS-ABC: Gaussian process surrogate approximate Bayesian computation. In: *Proceedings of the Thirtieth Conference on Uncertainty in Artificial Intelligence (UAI)*, pp 593–602
- Morris TP, White IR, Crowther MJ (2019) Using simulation studies to evaluate statistical methods. *Statistics in Medicine* 38(11):2074–2102. <https://doi.org/10.1002/sim.8086>
- Papamakarios G, Murray I (2016) Fast ϵ -free inference of simulation models with Bayesian conditional density estimation. In: *Advances in Neural Information Processing Systems* 29 (NeurIPS), pp 1028–1036
- Radev ST, Mertens UK, Voss A, et al (2020) BayesFlow: Learning complex stochastic models with invertible neural networks. *arXiv preprint arXiv:200306281*
- Talts S, Betancourt M, Simpson D, et al (2018) Validating Bayesian inference algorithms with simulation-based calibration. *arXiv preprint arXiv:180406788*
- Tejero-Cantero A, Boelts J, Deistler M, et al (2020) sbi: A toolkit for simulation-based inference.

Journal of Open Source Software 5(52):2505.
<https://doi.org/10.21105/joss.02505>

Wilkinson RD (2014) Accelerating ABC methods using Gaussian processes. In: Proceedings of the Seventeenth International Conference on Artificial Intelligence and Statistics (AISTATS), pp 1015–1023

Wood SN (2010) Statistical inference for noisy nonlinear ecological dynamic systems. Nature 466:1102–1104. <https://doi.org/10.1038/nature09319>